

Can you explain step by step and in detail how to test a chain mediation model, please?

ChatGPT said:

Testing a **chain mediation model** involves examining whether a predictor variable (X) influences an outcome variable (Y) through **two or more mediators in sequence** (e.g., $X \rightarrow M1 \rightarrow M2 \rightarrow Y$). Here's a **step-by-step and detailed explanation** of how to test such a model:

Step 1: Formulate the Hypothesis

Example hypothesis:

"X affects Y through two sequential mediators M1 and M2."

- Path a_1 : $X \rightarrow M1$
- Path a_2 : $M1 \rightarrow M2$
- Path b : $M2 \rightarrow Y$

So the **indirect path** is:

$X \rightarrow M1 \rightarrow M2 \rightarrow Y$

The **total effect** of X on Y is broken down into:

- Direct effect (c'): $X \rightarrow Y$ (controlling for M1 and M2)
 - Indirect effects:
 - $a_1 \times a_2 \times b$ (chain mediation)
 - $a_1 \times b_1$ (if M1 directly affects Y, optional depending on your model)
 - $a_2 \times b$ (if M2 mediates without M1, optional)
-

Step 2: Collect and Prepare Data

- Collect data for all four variables: X, M1, M2, and Y.
- Check for:
 - Normality
 - Outliers
 - Missing data
 - Linearity

You can use tools like SPSS, R, Python, or structural equation modeling (SEM) software (e.g., AMOS, Mplus, or lavaan in R).

Step 3: Specify the Model

Option A: Regression-Based Approach (e.g., using PROCESS macro in SPSS or Hayes' approach)

You'll run a series of regression models:

1. $M1 = \beta_0 + a_1 \cdot X + e_1$
(X predicts M1)
2. $M2 = \beta_0 + dX + a_2 M1 + e_2$
(X and M1 predict M2)
3. $Y = \beta_0 + c'X + b_1 M1 + b_2 M2 + e_3$
(X, M1, and M2 predict Y)

Then test the **indirect effect**:

$a_1 \times a_2 \times b$ (this is the chained mediation path)

Use bootstrapping (e.g., 5,000 resamples) to generate **confidence intervals** for this indirect effect.

Option B: SEM (Structural Equation Modeling)

- Specify a **path model**: $X \rightarrow M1 \rightarrow M2 \rightarrow Y$
- Estimate all paths simultaneously
- Use **fit indices** to assess model fit:
 - RMSEA (< .06)
 - CFI (> .95)
 - SRMR (< .08)
- Test indirect effects using **bootstrapping** or **model constraints**

Step 4: Test Indirect Effects

Use **bootstrapping** to test the significance of:

- The total indirect effect ($a_1 \times a_2 \times b$)
- Compare with other possible paths (if needed)

Bootstrapping is preferred because it does **not assume normality** of the sampling distribution of the indirect effect.

Step 5: Interpret Results

- If the **bootstrapped CI** for the indirect effect does **not include zero**, the chain mediation is **statistically significant**.
 - Evaluate the size of the effect (e.g., use standardized coefficients)
 - Consider alternative models (e.g., reverse paths, direct effects)
-

Optional: Control Variables and Model Comparison

- Include covariates if needed
 - Compare models with and without mediators to understand:
 - Total effect ($X \rightarrow Y$)
 - Direct effect ($X \rightarrow Y$, controlling for mediators)
 - Total indirect effect
-

Would you like a real example with data or syntax (e.g., in SPSS, R, or Python)?

You said:

Yes

ChatGPT said:

Great! Let's walk through a **real example** using **R with the lavaan package** (widely used for structural equation modeling), testing a **chain mediation model** with simulated data.

Example: Chain Mediation in R using lavaan

Hypothetical Scenario

Research question:

Does the effect of Self-Esteem (X) on Job Performance (Y) occur through Motivation (M1) and Job Satisfaction (M2), in that order?

So our model is:

Self-Esteem $\rightarrow$ Motivation $\rightarrow$ Job Satisfaction $\rightarrow$ Job Performance

Step 1: Install and Load Required Packages

r

CopyEdit

```
install.packages("lavaan") # if not installed  
library(lavaan)
```

Step 2: Simulate Example Data

```
r  
CopyEdit  
set.seed(123)  
  
n <- 300  
self_esteem <- rnorm(n)  
motivation <- 0.5 * self_esteem + rnorm(n, 0, 0.5)  
job_satisfaction <- 0.6 * motivation + rnorm(n, 0, 0.5)  
job_performance <- 0.7 * job_satisfaction + rnorm(n, 0, 0.5)  
  
data <- data.frame(  
  self_esteem,  
  motivation,  
  job_satisfaction,  
  job_performance  
)
```

Step 3: Specify the Chain Mediation Model

```
r  
CopyEdit  
model <- '  
  # Direct paths  
  motivation ~ a1*self_esteem  
  job_satisfaction ~ a2*motivation  
  job_performance ~ b*job_satisfaction + c_prime*self_esteem
```

```
# Indirect effect (chain)
```

```
indirect := a1 * a2 * b
```

```
# Total effect
```

```
total := c_prime + (a1 * a2 * b)
```

Step 4: Fit the Model Using SEM

r

CopyEdit

```
fit <- sem(model, data = data, se = "bootstrap", bootstrap = 5000)
```

```
summary(fit, standardized = TRUE, fit.measures = TRUE)
```

Step 5: Interpret the Results

Look for:

- **Path coefficients:** Are a1, a2, b significant?
 - **Indirect effect (indirect):** Look at Estimate, SE, and CI.
 - **If CI for indirect does not include 0 → significant mediation**
 - **Total effect:** Gives you overall influence of X on Y
-

Would you like me to provide the same example in SPSS (using PROCESS), or Python (e.g., using statsmodels or SEMpy)?

You said:

Yes, Python

ChatGPT said:

Great! Here's how to **test a chain mediation model in Python** using the **semopy** library, which allows you to specify structural equation models (including multiple mediators in sequence).

Chain Mediation in Python using semopy

Step 1: Install Required Libraries

bash

CopyEdit

```
pip install semopy pandas numpy
```

Step 2: Simulate Data

We'll use the same scenario:

Self-Esteem → Motivation → Job Satisfaction → Job Performance

python

CopyEdit

```
import numpy as np
```

```
import pandas as pd
```

```
np.random.seed(123)
```

```
n = 300
```

```
# Simulate variables
```

```
self_esteem = np.random.normal(size=n)
```

```
motivation = 0.5 * self_esteem + np.random.normal(0, 0.5, size=n)
```

```
job_satisfaction = 0.6 * motivation + np.random.normal(0, 0.5, size=n)
```

```
job_performance = 0.7 * job_satisfaction + np.random.normal(0, 0.5, size=n)
```

```
# Create DataFrame
```

```
data = pd.DataFrame({
```

```
    'self_esteem': self_esteem,
```

```
    'motivation': motivation,
```

```
    'job_satisfaction': job_satisfaction,
```

```
'job_performance': job_performance
})
```

Step 3: Specify the SEM Model

python

CopyEdit

```
from semopy import Model
```

```
model_desc = """
# Regressions
motivation ~ a1*self_esteem
job_satisfaction ~ a2*motivation
job_performance ~ b*job_satisfaction + c_prime*self_esteem

# Define indirect and total effects
indirect := a1 * a2 * b
total := c_prime + indirect
"""
```

Step 4: Fit the Model

python

CopyEdit

```
model = Model(model_desc)
```

```
res = model.fit(data)
```

```
# Print estimates
```

```
estimates = model.inspect()
```

```
print(estimates)
```

Step 5: Interpret Results

- Look at coefficients for a_1 , a_2 , b – should be significant
 - indirect gives the **chain mediation effect**
 - total gives the **total effect** of self_esteem on job_performance
 - Use bootstrap manually for CIs if needed (semopy does not bootstrap by default)
-

Would you like the bootstrapping part implemented as well in Python for confidence intervals?

You said:

Yes

ChatGPT said:

Great — let's add **bootstrapping** in Python to get **confidence intervals for the indirect effect** in the chain mediation model.

Since semopy doesn't natively support bootstrapping, we'll implement it manually by resampling the dataset multiple times, re-fitting the model each time, and extracting the indirect effect.

Step-by-Step: Bootstrapping the Indirect Effect

Required Libraries

python

CopyEdit

```
import numpy as np
```

```
import pandas as pd
```

```
from semopy import Model
```

Define the SEM Model (as before)

python

CopyEdit

```
model_desc = """
```

```
motivation ~ a1*self_esteem
```

```
job_satisfaction ~ a2*motivation
```



```
job_performance ~ b*job_satisfaction + c_prime*self_esteem
```

```
indirect := a1 * a2 * b
```

```
total := c_prime + indirect
```

```
"""
```

Fit Original Model and Extract Indirect Effect

python

CopyEdit

```
model = Model(model_desc)
```

```
model.fit(data)
```

```
original_indirect = model.inspect().loc['indirect', 'Estimate']
```

```
print(f"Original indirect effect: {original_indirect}")
```

Step 6: Bootstrap Confidence Intervals

python

CopyEdit

```
n_bootstraps = 1000
```

```
indirect_effects = []
```

```
for _ in range(n_bootstraps):
```

```
    sample = data.sample(n=len(data), replace=True)
```

```
    boot_model = Model(model_desc)
```

```
    try:
```

```
        boot_model.fit(sample)
```

```
        effect = boot_model.inspect().loc['indirect', 'Estimate']
```

```
        indirect_effects.append(effect)
```

```
    except:
```

```
        continue # Skip failed fits
```

Step 7: Compute Confidence Interval

python

CopyEdit

```
lower = np.percentile(indirect_effects, 2.5)
```

```
upper = np.percentile(indirect_effects, 97.5)
```

```
print(f"Bootstrap 95% CI for indirect effect: [{lower:.3f}, {upper:.3f}])")
```

```
# Optional: Check significance
```

```
if lower > 0 or upper < 0:
```

```
    print("Indirect effect is statistically significant.")
```

```
else:
```

```
    print("Indirect effect is not statistically significant.")
```

This gives you a **95% bootstrap confidence interval** for the **chained mediation effect**:

$X \rightarrow M1 \rightarrow M2 \rightarrow Y$

Would you like a full downloadable script, or want to visualize the mediation path as well?

You said:

both

ChatGPT said:

To run this Python-based chain mediation model with bootstrapping and visualization, you'll need to **log in to ChatGPT** to use Python code execution here. However, you can also run the full script locally on your own machine.

Here's the **complete Python code** you can copy and paste into a Jupyter Notebook or any Python IDE:

Full Script: Chain Mediation with Bootstrapping and Visualization

python

CopyEdit

```

import numpy as np
import pandas as pd
from semopy import Model
import matplotlib.pyplot as plt
import networkx as nx

# Simulate data
np.random.seed(123)
n = 300
self_esteem = np.random.normal(size=n)
motivation = 0.5 * self_esteem + np.random.normal(0, 0.5, size=n)
job_satisfaction = 0.6 * motivation + np.random.normal(0, 0.5, size=n)
job_performance = 0.7 * job_satisfaction + np.random.normal(0, 0.5, size=n)

data = pd.DataFrame({
    'self_esteem': self_esteem,
    'motivation': motivation,
    'job_satisfaction': job_satisfaction,
    'job_performance': job_performance
})

# Define SEM model
model_desc = """
motivation ~ a1*self_esteem
job_satisfaction ~ a2*motivation
job_performance ~ b*job_satisfaction + c_prime*self_esteem

indirect := a1 * a2 * b
total := c_prime + indirect

```

```
"""
```

```
# Fit model
```

```
model = Model(model_desc)
```

```
model.fit(data)
```

```
results = model.inspect()
```

```
original_indirect = results.loc['indirect', 'Estimate']
```

```
print(f"Original indirect effect: {original_indirect:.3f}")
```

```
# Bootstrapping
```

```
n_bootstraps = 1000
```

```
indirect_effects = []
```

```
for _ in range(n_bootstraps):
```

```
    sample = data.sample(n=len(data), replace=True)
```

```
    boot_model = Model(model_desc)
```

```
    try:
```

```
        boot_model.fit(sample)
```

```
        effect = boot_model.inspect().loc['indirect', 'Estimate']
```

```
        indirect_effects.append(effect)
```

```
    except:
```

```
        continue
```

```
# Confidence Interval
```

```
lower = np.percentile(indirect_effects, 2.5)
```

```
upper = np.percentile(indirect_effects, 97.5)
```

```
print(f"Bootstrap 95% CI for indirect effect: [{lower:.3f}, {upper:.3f}])")
```

```
if lower > 0 or upper < 0:
```

```
    print("Indirect effect is statistically significant.")
```

```

else:

    print("Indirect effect is not statistically significant.")


# Optional: Visualization of mediation path
G = nx.DiGraph()
edges = [
    ("Self-Esteem", "Motivation"),
    ("Motivation", "Job Satisfaction"),
    ("Job Satisfaction", "Job Performance"),
    ("Self-Esteem", "Job Performance") # direct effect
]
G.add_edges_from(edges)


plt.figure(figsize=(8, 5))
pos = {
    "Self-Esteem": (-1, 0),
    "Motivation": (0, 1),
    "Job Satisfaction": (1, 1),
    "Job Performance": (2, 0)
}
nx.draw(G, pos, with_labels=True, arrows=True, node_color='lightblue', node_size=2000,
font_size=10)

plt.title("Chain Mediation Path Diagram")
plt.show()

```

Let me know if you'd like a downloadable .py or Jupyter notebook file for convenience!